

Fichas de Transfer Learning

CIFAR-10, VGG-16 como feature extractor y con fine-tuning, y cómo medir lo que cuesta cada modelo. 89 fichas en 9 temas.

Brandon Werner Rivera Cabrera · 23088 · Universidad del Valle de Guatemala



Datos y preprocesamiento

12 fichas

¿Qué contiene CIFAR-10 y cómo viene dividido?

60 000 imágenes RGB de 32×32 en 10 clases: 50 000 de entrenamiento y 10 000 de test, con 6 000 por clase (5 000 + 1 000). Está perfectamente balanceado, así que accuracy y F1 macro cuentan casi la misma historia.

¿Cuáles son las 10 clases?

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. Son 6 animales y 4 vehículos. Automobile incluye sedanes y SUVs, truck solo camiones grandes, y las pickups no están en ninguna de las dos.

¿Qué pares de clases son difíciles y por qué?

cat ↔ dog: cuerpo, pelaje y pose parecidos a 32×32. **automobile ↔ truck**: vehículos con ruedas vistos de lado. **deer ↔ horse**: cuadrúpedos cafés sobre pasto. **airplane ↔ bird ↔ ship**: un objeto pequeño sobre cielo o agua, donde el fondo domina la imagen.

Media y desviación por canal: CIFAR-10 frente a ImageNet

	MEDIA (R, G, B)	STD (R, G, B)
CIFAR-10	0.491, 0.482, 0.447	0.247, 0.243, 0.262
ImageNet	0.485, 0.456, 0.406	0.229, 0.224, 0.225

Son cercanas; CIFAR tiene más azul por los fondos de cielo y agua. Los valores 0.2023, 0.1994, 0.2010 que circulan en internet son el promedio de la std de cada imagen, no la std del dataset.

¿Con qué estadísticas se normaliza cada modelo?

La CNN desde cero usa media y std de CIFAR-10 calculadas solo con el split de entrenamiento, para no filtrar información de validación ni de test. VGG-16 usa las de ImageNet, porque sus filtros se aprendieron con entradas normalizadas así; cambiarlas mueve la distribución que espera la primera capa.

¿Para qué sirve el split estratificado 45 000 / 5 000?

Validación sirve para elegir hiperparámetros y el mejor epoch. Estratificar deja exactamente 500 imágenes por clase en validación, así la métrica no depende de qué clases cayeron ahí. El test oficial de 10 000 queda intacto.

```
train_test_split(idx, test_size=5000, stratify=y, random_state=42)
```

Transfer learning

10 fichas

¿Qué es transfer learning?

Reutilizar los pesos de un modelo entrenado en una tarea fuente grande, como ImageNet con 1.28 M imágenes y 1 000 clases, como punto de partida para otra tarea. Se transfieren las representaciones aprendidas, no los datos.

Feature extractor, fine-tuning y desde cero: ¿qué se entrena en cada uno?

Feature extractor: backbone congelado; solo se entrena la cabeza nueva. **Fine-tuning**: se descongelan los bloques superiores y se entrenan con un lr pequeño junto con la cabeza. **Desde cero**: todos los pesos empiezan aleatorios y se entrenan.

¿Por qué se congelan las capas bajas y se ajustan las altas?

Las capas bajas detectan bordes, colores y texturas que sirven para casi cualquier imagen. Las altas combinan eso en partes de objetos específicas de ImageNet. La transferibilidad baja con la profundidad (Yosinski et al., 2014), así que lo que más conviene adaptar son los bloques superiores.

¿Cuándo conviene cada estrategia?

	DOMINIO PARECIDO	DOMINIO DISTINTO
Pocos datos	Feature extractor	FE sobre capas intermedias o fine-tuning muy suave
Muchos datos	Fine-tuning	Fine-tuning profundo o desde cero

Si la inferencia tiene que ser muy barata, se prefiere un modelo pequeño, preentrenado liviano o entrenado desde cero.

¿Qué es el olvido catastrófico en fine-tuning?

Con un lr alto, los gradientes grandes de una cabeza aleatoria destruyen los pesos preentrenados antes de que la cabeza aprenda algo útil, y el modelo pierde las features que justificaban usarlo. Se evita con lr bajo en el backbone, warmup, pocos bloques descongelados o entrenando primero la cabeza.

VGG-16 por dentro

13 fichas

¿De dónde viene el nombre VGG-16?

Del Visual Geometry Group de Oxford (Simonyan y Zisserman, 2014). El 16 cuenta las capas con pesos: 13 convolucionales y 3 lineales.

¿Cómo están organizados los 5 bloques convolucionales?

Convs por bloque: 2, 2, 3, 3, 3. Canales: 64, 128, 256, 512, 512. Todas son conv 3×3 con padding 1 seguidas de ReLU, y cada bloque cierra con MaxPool 2×2 de stride 2, así que la resolución se divide entre 2 por bloque y entre 32 en total.

¿Por qué el test se usa una sola vez?

Si eliges modelo, epoch o hiperparámetros mirando el test, este se vuelve otro conjunto de validación y su métrica queda optimista por sesgo de selección. Por eso solo el mejor modelo de cada familia se evalúa en test, al final.

¿Qué augmentation se usa y por qué solo en entrenamiento?

Recorte aleatorio con relleno de 4 píxeles y volteo horizontal. En train genera variantes plausibles de cada imagen y regulariza. En validación y test la entrada debe ser determinista y de la distribución real; si no, la métrica tiene ruido y dos iteraciones dejan de ser comparables.

¿Por qué no se usa volteo vertical?

Porque produce imágenes que no existen en el dataset: barcos, caballos o aviones de cabeza. La augmentation solo debe aplicar transformaciones que no cambian la clase ni sacan la imagen de la distribución de test.

¿Cuánto crece el trabajo al pasar de 32×32 a 128×128 y a 224×224?

ENTRADA	VGG-16 GMACS	SALIDA DE FEATURES
32×32	0.43	512×1×1
64×64	1.37	512×2×2
128×128	5.13	512×4×4
224×224	15.47	512×7×7

Los píxeles crecen $128^2/32^2 = 16\times$ y $224^2/32^2 = 49\times$. El cómputo de las convoluciones crece igual, con el área de la imagen.

¿Redimensionar de 32 a 128 agrega información a la imagen?

No. La interpolación reparte los mismos 1 024 píxeles y la imagen queda borrosa. Lo que cambia es la escala: los objetos ocupan un tamaño parecido al que los filtros vieron en ImageNet y el mapa final de VGG deja de colapsar a 1×1.

¿Qué hace el pipeline de `weights.transforms()` de VGG-16?

Resize a 256, CenterCrop a 224, conversión a float en [0, 1] y Normalize con media y std de ImageNet, con interpolación bilineal. Es la receta con la que se midió el 71.6 % top-1. En CIFAR se omite el center crop porque el objeto ya llena la imagen y se perderían bordes; se redimensiona directo al tamaño final.

¿Por qué el backbone usa un lr 10 veces menor que el clasificador?

Los pesos preentrenados ya están cerca de una buena solución y solo necesitan ajustes pequeños; la cabeza nueva es aleatoria y necesita pasos grandes. A esto se le llama learning rates discriminativos. En FT-1 a FT-3: backbone 1e-5 y clasificador 1e-4.

¿Qué se gana y qué se arriesga al descongelar más bloques?

Se gana capacidad para adaptar las features al dominio. Se paga con más parámetros entrenables, un backward más caro, más memoria y más riesgo de sobreajuste y olvido. Por eso FT-1, FT-2 y FT-3 solo cambian el número de bloques descongelados: 5; 4 y 5; 3, 4 y 5.

¿Por qué las features de ImageNet sirven con imágenes de 32×32 reescaladas?

Los bordes, colores y texturas siguen ahí, y casi todas las clases de CIFAR tienen equivalentes en ImageNet: perros, gatos, aves, barcos, camiones. El reescalado acerca los objetos a la escala que vieron los filtros. La imagen borrosa sí es un cambio de dominio, y por eso el fine-tuning supera al feature extractor.

¿Por qué el fine-tuning suele superar al feature extractor?

El feature extractor solo puede separar lo que las features de ImageNet ya separan. El fine-tuning ajusta las capas altas a la baja resolución y a las diferencias finas de CIFAR, como gato contra perro.

Con solo el 10 % de los datos, ¿qué modelo se espera que caiga más?

La CNN desde cero, porque debe aprender todas sus features con 4 500 imágenes. El feature extractor es el más estable porque solo aprende la cabeza. El fine-tuning queda en medio: conserva la ventaja del preentrenamiento, pero con pocos datos sus millones de parámetros entrenables sobreajustan más rápido. Confírmalo con tu tabla del experimento 4.1.

¿Qué índices de `model.features` ocupa cada bloque?

Bloque 1 `[8:5]`, bloque 2 `[5:10]`, bloque 3 `[10:17]`, bloque 4 `[17:24]`, bloque 5 `[24:31]`. Son 31 módulos entre convs, ReLUs y MaxPools. Para descongelar desde el bloque 4:

```
model.features[17:].requires_grad_(True)
```

¿Qué contiene `model.classifier`?

Linear(25088, 4096) → ReLU → Dropout(0.5) → Linear(4096, 4096) → ReLU → Dropout(0.5) → Linear(4096, 1000). El índice 6 es la capa que se reemplaza por Linear(4096, 10).

¿Qué hace `model.avgpool`?

Es AdaptiveAvgPool2d((7, 7)): entrega siempre un mapa de 7×7 sin importar el tamaño de entrada, así la primera lineal recibe 512·7·7 = 25 088 valores. Gracias a eso VGG-16 acepta 128×128 sin tocar el clasificador.

¿Qué forma tiene la salida de `features` según la entrada?

Entrada entre 32, con 512 canales: 224 → 7×7, 128 → 4×4, 64 → 2×2, 32 → 1×1. Con 32×32 el avgpool copia el único valor 49 veces, y el clasificador recibe 25 088 números de los que solo 512 son distintos.

¿Cuántos parámetros tiene VGG-16 y dónde están?

PARTE	PARÁMETROS	%
features (13 convs)	14 714 688	11
classifier (cabeza de 10)	119 586 826	89
solo Linear(25088, 4096)	102 764 544	77
Total con 10 clases	134 301 514	100

Con la cabeza original de 1 000 clases son 138 357 544.

¿Dónde está el cómputo de VGG-16?

En las convoluciones. A 128×128, features hace 5.01 GMACs y classifier 0.12 GMACs. Parámetros y cómputo están en lugares opuestos: el clasificador tiene el 89 % de los parámetros y hace el 2 % del trabajo.

● PyTorch

¿Cómo se carga VGG-16 preentrenada con su preprocesamiento?

Con `weights=None` se obtiene la misma arquitectura con pesos aleatorios. `VGG16_Weights.DEFAULT` apunta a IMAGENET1K_V1.

```
from torchvision.models import vgg16, VGG16_Weights
weights = VGG16_Weights.IMAGENET1K_V1
model = vgg16(weights=weights)
preprocess = weights.transforms()
```

¿Cómo se congela el backbone y se pone una cabeza de 10 clases?

El orden importa: si congelas después de reemplazar la capa, también congelas la cabeza nueva y el modelo no aprende nada.

```
model.requires_grad_(False)
model.classifier[6] = nn.Linear(4096, 10) # nace con requires_grad=True
```

¿Qué hace exactamente `requires_grad=False`?

Autograd no calcula el gradiente de ese parámetro, `p.grad` queda en None y el optimizador no lo mueve. El forward es idéntico. Si ninguna capa por debajo necesita gradiente, el backward se detiene en la capa entrenable más baja, y ese es el ahorro real.

`model.train()` frente a `model.eval()`
Solo cambian el comportamiento de algunas capas. **Dropout**: activo en train, identidad en eval. **BatchNorm**: en train usa la media y varianza del lote y actualiza sus running stats; en eval usa las running stats fijas. Ninguno de los dos toca los gradientes.

`torch.no_grad()` frente a `model.eval()`
`no_grad` no construye el grafo de autograd: ahorra memoria de activaciones y tiempo, pero no cambia dropout ni BN. `eval()` hace lo contrario. Para evaluar se usan los dos. `torch.inference_mode()` es una versión más estricta y algo más rápida de `no_grad`.

```
model.eval()
with torch.no_grad():
    logits = model(x)
```

¿Qué pasa si evalúas sin llamar a `eval()`?

Dropout sigue apagando neuronas al azar y BatchNorm usa las estadísticas del lote y además las modifica. Las métricas salen más bajas, ruidosas y dependientes del tamaño de batch.

● Costo computacional

¿Qué diferencia hay entre MACs y FLOPs?

Un MAC es una multiplicación más una suma acumulada, es decir 2 FLOPs, así que GFLOPs = 2 × GMACs. Muchos papers y librerías dicen FLOPs cuando reportan MACs: el 15.47 que torchvision lista como GFLOPs de VGG-16 son en realidad GMACs.

MACs y parámetros de una convolución

MACs = $H_{out} \cdot W_{out} \cdot C_{out} \cdot C_{in} \cdot k^2$. Parámetros = $C_{out} \cdot (C_{in} \cdot k^2 + 1)$. La primera conv de VGG a 224×224 hace 224·224·64·3·9 = 86.7 M MACs con solo 1 792 parámetros, porque cada peso se reutiliza en cada posición.

Parámetros y GMACs de cada bloque a 128×128

BLOQUE	PARÁMETROS	GMACS
1 (64)	38 720	0.63
2 (128)	221 440	0.91
3 (256)	1 475 328	1.51
4 (512)	5 899 776	1.51
5 (512)	7 079 424	0.45

Los bloques altos tienen los parámetros y los bajos el cómputo, porque trabajan a mayor resolución.

Parámetros entrenables en cada iteración de VGG-16

ITERACIÓN	QUÉ SE ENTRENA	ENTRENABLES
FE-1	solo Linear(4096, 10)	40 970
FE-2, FE-3	clasificador completo	119 586 826
FT-1	bloque 5 + clasificador	126 666 250
FT-2, FT-4	bloques 4-5 + clasificador	132 566 026
FT-3	bloques 3-5 + clasificador	134 041 354

¿Qué pesos se usan y qué tan buenos son en ImageNet?

`VGG16_Weights.IMAGENET1K_V1`: 71.59 % top-1 y 90.38 % top-5, en un archivo de 528 MB. Existe también `IMAGENET1K_FEATURES`, que solo trae pesos válidos en features, y `vgg16_bn` con BatchNorm, que llega a 73.36 % top-1.

¿Por qué VGG apila convoluciones 3×3?

Dos 3×3 seguidas ven una región de 5×5 y tres ven 7×7, con menos parámetros (3·9·C² = 27C² frente a 49C² de una sola 7×7) y una ReLU entre cada una, lo que agrega no linealidad.

13 fichas

¿Cómo se usa un lr distinto para el backbone y para el clasificador?

Con grupos de parámetros. Cada grupo guarda su lr en `opt.param_groups[i]["lr"]` y los schedulers escalan todos los grupos a la vez.

```
opt = torch.optim.AdamW([
    {"params": [p for p in model.features.parameters() if p.requires_grad], "lr": 1e-5},
    {"params": model.classifier.parameters(), "lr": 1e-4},
], weight_decay=1e-4)
```

¿Qué parámetros se le pasan al optimizador?

Solo los entrenables: `[p for p in model.parameters() if p.requires_grad]`. Pasar congelados no falla, porque su grad es None y se saltan, pero filtrarlos deja explícito qué se está entrenando.

¿Qué espera `nn.CrossEntropyLoss`?

Logits sin softmax de forma (N, C) y etiquetas enteras de forma (N,). Aplica log-softmax y NLL por dentro. Poner un softmax al final del modelo aplanar los gradientes y empeora el entrenamiento.

¿Cómo se cuentan los parámetros totales y los entrenables?

```
total = sum(p.numel() for p in model.parameters())
entrenables = sum(p.numel() for p in model.parameters() if p.requires_grad)
```

¿Qué hace `torch.autocast` con `bfloat16`?

Ejecuta convs y matmuls en bf16 y mantiene los pesos en fp32. bf16 tiene el mismo rango de exponente que fp32, así que los gradientes no se desbordan y no hace falta GradScaler. fp16 sí lo necesita.

```
with torch.autocast("cuda", dtype=torch.bfloat16):
    loss = criterio(model(x), y)
```

¿Para qué sirven `channels_last` y `cuda.benchmark`?

`channels_last` guarda los tensores como NHWC, el formato que aprovechan los Tensor Cores en convoluciones. `cuda.benchmark` prueba varios algoritmos de convolución la primera vez que ve una forma de entrada y se queda con el más rápido; conviene cuando la forma no cambia entre lotes.

¿Qué es `AdaptiveAvgPool2d`?

Un pooling que recibe el tamaño de salida en lugar del kernel y calcula las ventanas según la entrada. `AdaptiveAvgPool2d(1)` es global average pooling, la cabeza de la CNN propia; `AdaptiveAvgPool2d((7, 7))` es el de VGG-16.

13 fichas

MACs de una capa lineal

in × out por imagen, uno por peso. Linear(25088, 4096) hace 102.8 M MACs y tiene 102.8 M parámetros: una lineal usa cada peso una sola vez, una conv lo reutiliza en cada posición del mapa.

¿Qué reporta cada herramienta de conteo?

HERRAMIENTA	QUÉ DEVUELVE
torchinfo.summary	parámetros totales y entrenables, forma por capa y Mult-Adds (MACs)
thop.profile	(MACs, parámetros)
ptflops	MACs y parámetros
fvcore FlopCountAnalysis	dice flops pero cuenta un multiply-add como 1: son MACs
torch.utils.flop_counter	FlopCounterMode cuenta FLOPs reales, 2 por MAC

¿Cómo se estiman los FLOPs de entrenamiento por imagen?

Forward $\approx$ 2-MACs FLOPs. Backward $\approx$ 2 veces el forward, porque calcula gradientes respecto a las activaciones y respecto a los pesos. Con toda la red entrenable: $\approx$ 6-MACs. Con partes congeladas: $\approx$ 2-MACs_{total} + 4-MACs_{entrenables}, porque el backward solo recorre las capas desde la entrenable más baja. El total del entrenamiento es eso $\times$ imágenes $\times$ epochs.

¿Por qué el feature extractor es tan barato por epoch si entrena 119 M parámetros?

Su backward solo recorre el clasificador, que a 128 $\times$ 128 hace 0.12 GMACs, y nunca entra a las convoluciones, que hacen 5.01. El costo del backward depende de los MACs de las capas entrenables, no de cuántos parámetros tienen.

¿Qué truco hace todavía más barato al feature extractor?

Si el backbone está congelado y no hay augmentation, la salida de features de cada imagen nunca cambia: se calcula una vez, se guarda y se entrena la cabeza sobre esos vectores, y cada epoch cuesta casi nada. Con augmentation deja de servir, porque cada epoch ve imágenes distintas.

Entrenamiento y regularización

¿Qué hace BatchNorm y dónde va en la CNN?

Normaliza cada canal con la media y varianza del lote y luego aplica una escala γ y un desplazamiento β aprendidos. Estabiliza el entrenamiento y permite lr altos. Va después de la conv, que se declara con `bias=False` porque β ya cumple esa función. En eval usa las running stats.

Dropout frente a Dropout2d

Dropout apaga neuronas individuales con probabilidad p y escala las demás por $1/(1-p)$ durante train. **Dropout2d** apaga canales completos de un mapa de activación; en convs funciona mejor porque los píxeles vecinos están correlacionados y apagar uno suelto casi no regulariza.

Adam con weight_decay frente a AdamW

Adam suma λw al gradiente y luego lo divide por la raíz del segundo momento, así los pesos con gradientes grandes casi no se regularizan. AdamW aplica el decaimiento directo al peso, desacoplado del gradiente: $w \leftarrow w - lr \cdot \lambda \cdot w$. AdamW regulariza como se espera.

¿Cómo funciona OneCycle con SGD?

El lr sube desde `max_lr/25` hasta `max_lr` durante el primer 25 % del entrenamiento y luego baja con coseno casi hasta 0; el momentum hace el recorrido inverso. Con BatchNorm permite lr de 0.1 y converger en pocos epochs. Lo usa CNN-3.

Evaluación

Precision, recall y F1 de una clase

Precision = $TP / (TP + FP)$: de lo que predije como c , cuánto era c . **Recall** = $TP / (TP + FN)$: de lo que era c , cuánto encontré. **F1** = $2 \cdot PR / (P + R)$, la media armónica, que castiga cuando una de las dos es baja.

Promedio macro, weighted y micro

Macro: promedio simple de las 10 clases, cada clase pesa igual. **Weighted**: ponderado por el número de ejemplos de cada clase. **Micro**: se calcula sobre todos los ejemplos juntos y, en multiclase con una etiqueta, es igual a la accuracy. En CIFAR-10 balanceado macro y weighted casi coinciden.

```
precision_recall_fscore_support(y, pred, average="macro")
```

¿Cómo se lee la matriz de confusión de sklearn?

Filas = clase real, columnas = clase predicha, y la diagonal son los aciertos. Normalizar por filas da el recall de cada clase. Un valor alto en la fila cat, columna dog, son gatos clasificados como perros.

Despliegue

¿Qué modelo desplegar en un servidor con GPU?

El de mejor métrica, normalmente VGG-16 con fine-tuning, siempre que cumpla la latencia pedida. El servidor tiene memoria de sobra y puede agrupar peticiones en lotes para subir el throughput. Se acelera con bf16 o fp16, `torch.compile` o exportando a ONNX y TensorRT.

¿Por qué VGG-16 no es buena opción para un celular?

Son 134 M parámetros, unos 537 MB en fp32, y de 5 a 15 GMACs por imagen: mucha latencia, batería y memoria, y una app enorme. La CNN-2, con 1.15 M parámetros y 0.15 GMACs, sí cabe.

¿Cómo se mide bien la latencia en GPU?

CUDA es asíncrono: la llamada regresa antes de que el kernel termine. Hay que hacer warmup, sincronizar antes de arrancar y antes de detener el reloj, y promediar muchas repeticiones con batch 1.

```
for _ in range(10): model(x) # warmup
torch.cuda.synchronize(); t0 = time.perf_counter()
for _ in range(100): model(x)
torch.cuda.synchronize()
ms = (time.perf_counter() - t0) * 1000 / 100
```

Latencia frente a throughput

Latencia: tiempo de una imagen con batch 1, lo que siente un usuario. **Throughput**: imágenes por segundo con batch grande, lo que le importa a un servidor. Agrupar en lotes sube el throughput pero cada petición espera más.

¿Cómo se mide la memoria pico de GPU?

`max_memory_allocated` cuenta los tensores vivos. `memory_reserved` incluye la memoria que el caching allocator guarda para reutilizar y se parece más a lo que muestra nvidia-smi.

```
torch.cuda.reset_peak_memory_stats()
# ... entrenar ...
pico_mb = torch.cuda.max_memory_allocated() / 2**20
```

¿Para qué sirve el warmup antes del coseno?

Al inicio la cabeza es aleatoria y produce gradientes grandes. Empezar con un lr pequeño evita que esos primeros pasos dañen el backbone preentrenado. Después el coseno baja el lr suavemente hasta 0.

¿Cómo se ve el sobreajuste en las curvas de pérdida?

La pérdida de train sigue bajando mientras la de validación se estanca o sube, y la brecha entre ambas crece. Se combate con augmentation, dropout, weight decay, menos capacidad o quedándose con el mejor epoch.

¿Qué efecto tiene la augmentation en las curvas?

La pérdida de train queda más alta que sin augmentation, porque cada epoch ve versiones nuevas y más difíciles, pero la de validación baja más y tarda en subir. La brecha se achica.

¿Por qué se guarda el mejor epoch según el F1 de validación?

Es early stopping sin detener el entrenamiento: se conserva el estado que mejor generaliza aunque después sobreajuste. Los epochs hasta el mejor muestran qué tan rápido converge cada modelo.

¿Por qué la CNN usa global average pooling y no flatten más una lineal grande?

La cabeza queda en una sola Linear(C, 10), con muy pocos parámetros y menos sobreajuste, y además no depende de la resolución. VGG-16 hace lo opuesto y pone 102.8 M parámetros en su primera lineal.

¿Cómo saber si los tres modelos cometen los mismos errores?

Compara los pares fuera de la diagonal más grandes de cada matriz. Si los tres fallan en cat $\leftrightarrow$ dog, el límite está en los datos y la resolución más que en la arquitectura. Si solo la CNN falla en airplane $\leftrightarrow$ ship, ese error lo corrige el preentrenamiento.

¿Por qué graficar el F1 de test contra el costo en escala logarítmica?

Los costos difieren en órdenes de magnitud: la CNN-2 hace 0.15 GMACs por imagen y VGG-16 a 128 $\times$ 128 hace 5.13, unas 33 veces más, y en FLOPs de entrenamiento la diferencia es mayor. En escala lineal los puntos de la CNN quedarían aplastados contra el eje.

¿Qué preguntas responde la tabla comparativa final?

Qué modelo gana en test, cuánto cuesta cada punto de F1 en parámetros, MACs, latencia, memoria y tiempo, cuántos epochs necesita cada uno para llegar a su mejor punto y cuánto cae con el 10 % de los datos. La conclusión balancea rendimiento y recursos, no solo accuracy.

¿Qué arquitecturas se usan en móviles?

MODELO	PARAMS	GNACS	TOP-1
MobileNetV3-Small	2.5 M	0.06	67.7 %
MobileNetV2	3.5 M	0.30	72.2 %
MobileNetV3-Large	5.5 M	0.22	75.3 %
EfficientNet-B0	5.3 M	0.39	77.7 %
ResNet-18	11.7 M	1.81	69.8 %
VGG-16	138.4 M	15.47	71.6 %

Cifras de torchvision a 224 $\times$ 224 con sus mejores pesos. Todas tienen pesos de ImageNet, así que el mismo transfer learning aplica.

¿Por qué una convolución separable en profundidad es tan barata?

Divide la conv en una depthwise $k \times k$, un filtro por canal ($k^2 C_{in}$ MACs por posición), y una pointwise 1×1 que mezcla canales ($C_{in} C_{out}$). Frente a la conv normal ($k^2 C_{in} C_{out}$) cuesta $1/C_{out} + 1/k^2$ veces: con 3×3 , entre 8 y 9 veces menos. Es la base de MobileNet.

● Mi laboratorio

7 fichas

¿Por qué VGG-16 se entrenó a 128×128 y no a 224×224?

Cuesta 3 veces menos, 5.13 frente a 15.47 GMACs por imagen, y el mapa final queda de 4×4 , que todavía conserva información espacial; a 32×32 colapsaría a 1×1 . Se usa la misma resolución en el feature extractor y en el fine-tuning para que la comparación entre ambos sea justa, como pide el enunciado.

¿Por qué el preprocesamiento se hace en la GPU y no con DataLoader?

En Windows, DataLoader con `num_workers > 0` crea procesos con spawn, y dentro de Jupyter los procesos hijos no pueden importar funciones definidas en el notebook. Además, redimensionar 45 000 imágenes a 128×128 en CPU cada epoch sería el cuello de botella. Todo CIFAR-10 en uint8 ocupa 184 MB, así que cabe en la GPU.

¿Cómo se hacen el recorte y el volteo como una transformación afín?

Con `affine_grid` y una matriz $\theta = \begin{bmatrix} s & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$ por imagen, donde $s = \pm 1$ es el volteo y t_x, t_y son desplazamientos de hasta 4 píxeles en coordenadas normalizadas ($4/2/32$). `grid_sample` con relleno de ceros genera la imagen desplazada, volteada y ya redimensionada al tamaño de salida en una sola operación.

¿Qué cambia entre las iteraciones de cada modelo?

MODELO	QUÉ VARÍA
CNN	tamaño (3 bloques 32-128, 3 bloques 64-256, 4 bloques 64-512), augmentation con dropout, optimizador y schedule
FE	solo la última capa o el clasificador completo, con o sin augmentation
FT	bloques descongelados (5; 4-5; 3-5) y lr del backbone, 1e-5 o 5e-5 en FT-4

Cifras clave

138.36 M parámetros de VGG-16 con la cabeza de 1 000 clases	89 % de esos parámetros están en el clasificador	15.47 GMACs VGG-16 por imagen a 224×224	5.13 GMACs VGG-16 por imagen a 128×128
71.59 % top-1 en ImageNet de IMAGENET1K_V1	45 000 · 5 000 · 10 000 train, validación y test	16× y 49× más píxeles de 32 a 128 y de 32 a 224	1 MAC = 2 FLOPs entrenar cuesta ≈ 6 MACs por imagen

Cifras calculadas con torchvision 0.29 sobre la arquitectura real: parámetros con `numel()` y MACs con hooks en cada Conv2d y Linear. Las preguntas sobre resultados esperados se confirman con las tablas de tu notebook.

¿Qué técnicas reducen un modelo para desplegarlo?

Cuantización a INT8, que lo hace 4 veces más pequeño y más rápido en CPU y NPU; poda de pesos o canales; destilación hacia un modelo pequeño; y exportar a un runtime móvil como ExecuTorch, TensorFlow Lite, Core ML u ONNX Runtime Mobile.

FLOPs de entrenamiento por imagen de cada iteración

ITERACIÓN	ENTRADA	GFLOPS POR IMAGEN
CNN-1	32	0.23
CNN-2	32	0.92
CNN-3	32	1.25
FE-1	128	10.26
FE-2, FE-3	128	10.74
FT-1	128	12.55
FT-2, FT-4	128	18.59
FT-3	128	24.63

Fórmula: $2 \cdot \text{MACs}_{\text{total}} + 4 \cdot \text{MACs}_{\text{entrenables}}$. El total multiplica esto por 45 000 imágenes y por los epochs.

¿Qué diferencia hay entre FE-1 y FE-2?

FE-1 solo entrena la última lineal: es un clasificador lineal sobre las features de la penúltima capa, que ya están afinadas para las 1 000 clases de ImageNet. FE-2 reentrena las tres lineales y puede reorganizar esas features para CIFAR-10, a cambio de 119.6 M parámetros entrenables en lugar de 41 mil.

¿Qué archivos genera el notebook y qué hay en cada uno?

ARCHIVO	CONTENIDO
results/iteraciones.csv	una fila por iteración: configuración, métricas de validación, tiempo, memoria
results/historial.csv	pérdida y métricas por epoch
results/test.json	métricas de test, matrices y pares más confundidos
results/datos_10pct.csv	experimento con el 10 % de los datos
results/comparacion.csv	tabla de rendimiento y recursos
figs/	todas las figuras del informe